

Progettazione di Applicazioni Mobili

Flutter

Introduction

Stefano Burigat

Dipartimento di Scienze Matematiche, Informatiche e
Fisiche - Università di Udine

stefano.burigat@uniud.it



Flutter history

- Flutter is an SDK for application development, announced by Google in 2015 and released as version 1.0 in 2018
- Initially, it ran only on Android, was then ported to iOS, and lately to the Web and desktop platforms (Linux, Mac, Windows)
- Flutter uses Dart as programming language, created by Google in 2011
- Dart is a fully object-oriented, garbage-collected, C-based syntax style language with a sound typing system and language features like interfaces, mixins, abstract classes, reified generics, and static typing

Flutter design

- One of the main goals of Flutter was being able to render app UIs at a consistent 120fps
- Flutter renders its own UI components, it does not use native platform components
 - Flutter apps can be consistent across platforms
 - New UI components, or widgets, can be added to Flutter quickly and easily
 - Flutter provides design-specific widgets (Material design widgets and Cupertino design widgets)

Flutter structure

- Flutter main parts:
 - the Dart platform
 - the Flutter engine: a (mostly) C++-based codebase that uses the Impeller or Skia graphics engines to do its rendering
 - the foundation library: an interface over the native SDKs of native platforms
 - the widgets

Widgets

- In Flutter, almost everything (related to the UI of your app) is a widget
- A widget is a piece of UI and UIs are built by composing widgets (not all widgets manifest on the screen)
- Typical widgets in a UI: buttons, lists, images, text form fields, ...
- Also widgets: the padding around an image, the state of a text form field, the text displayed on the screen, the theme an app is using, the animations of a page, ...

Widgets

- In Flutter, your code is typically a giant hierarchy of widgets, the “widget tree”
 - most widgets are containers, meaning they can have one or more children, that can each have one or more children and so on
- All widgets are Dart classes, and widgets typically have only a single concrete requirement: they must supply a `build()` method that returns a widget (most of the times)
- Widgets usually can be built using two standard classes, `StatelessWidget` and `StatefulWidget`

StatelessWidget

- A widget that extends StatelessWidget never changes and is called a stateless widget because it has no state
- Things like Icon widgets, which display small images, and Text widgets, which display strings of text, are said to be stateless widgets

```
class MyTextWidget extends StatelessWidget {  
  
    Widget build(BuildContext) {  
        return new Text("Hello!");  
    }  
  
}
```

StatefulWidget

- The StatefulWidget base class has the notion of state in it, that is, it changes in some way when the user interacts with it
- A CheckBox, a Slider, a TextField, are all well-known examples of stateful widgets
- When you code such a widget, you actually have to create two classes: the stateful widget class itself, and a state class to go along with it
- When the state of a stateful widget changes, Flutter re-renders the portion of the screen where the widget resides (if necessary)

StatefulWidget

```
class LikesWidget extends StatefulWidget {  
    @override  
    LikesWidgetState createState() => LikesWidgetState();  
}  
  
class LikesWidgetState extends State<LikesWidget> {  
    int likeCount = 0;  
  
    void like() {  
        setState(() {  
            likeCount += 1;  
        });  
    }  
  
    @override  
    Widget build(BuildContext inContext) {  
        return Row(  
            children : [  
                RaisedButton(  
                    onPressed : like,  
                    child : Text('$likeCount')  
                )  
            ]  
        );  
    }  
}
```

StatelessWidget vs StatefulWidget

